iRace — engineering & codebase study guide

Real-app checklist, how this repo maps to it, audit changelog, and architecture notes — one document for offline study.

Generated 2026-04-09 · Source: `docs/*.md` in the iRace repository

Real applications & engineering mindset — a guide anchored in iRace

This document is for **juniors** who want to connect textbook ideas to a **real codebase**: what "production-shaped" software usually includes, **how iRace implements each piece** (where to look in the tree), **why order and boundaries matter**, and **how to think** when you start a project alone or join a team.

How to use it

- Read **Part A** once to build a mental checklist of "what grown-up apps worry about."
- Use **Part B** as a **map**: when you care about a topic, open the listed paths and read the code there (this guide explains *what role* each area plays; it does not paste implementation).
- Use **Parts C–E** to align your habits with how teams work and how this repo is structured.
- Cross-check `docs/AUDIT_CHANGES.md` for a chronological log of refactors already done on the frontend, and `docs/ARCHITECTURE.md` for a short technical index.

Exporting to PDF

- If you want a portable copy, render this Markdown to PDF the same way you did for the audit notes (e.g. print from a browser or your preferred toolchain). The **source of truth** stays this `.md` file in Git.

Part A — What a "real" application usually needs

A "real" app is not one magic stack. It is a **bundle of responsibilities** that teams discover over time. Below is a **practical checklist** you can reuse for any product. None of these are "only senior" topics — juniors benefit from **knowing the list**; seniors differ mainly in **depth of tradeoffs** and **operational scars**.

A1. Problem & boundaries

- **What it is:** Clear idea of **who the user is**, **what job the app does**, and **what is explicitly out of scope**.
- **Why it matters:** Prevents infinite features and guides every folder and API name.
- **In iRace:** A **Strava-backed fitness challenge** with invite links, leaderboards, and compliance-friendly consent flows — not a generic social network.

A2. Runtime topology (where code runs)

- **What it is:** Deciding **browser vs server vs database vs third-party** and **how they talk** (HTTP, webhooks, Web Push).
- **Why it matters:** Secrets, performance, and security follow from this split.
- **Pattern:** SPA + serverless API is common; "monolith + server-rendered HTML" is another valid pattern.

A3. Client application (UI)

- **What it is:** Screens, navigation, loading/error states, accessibility basics, responsive layout.
- **Why it matters:** This is what users judge first.
- **Engineering extras:** Component boundaries, shared UI primitives, global providers (auth, toast).

A4. Identity, sessions, and authorization

- **What it is: Who is this request?** Is the session valid? **What may this user do** on this challenge?
- **Why it matters:** Most business rules and abuse risk live here.
- **Note:** "OAuth with Strava" solves **delegated access to Strava**; your **app session** is still a separate design (tokens, cookies, expiry).

A5. Data model & persistence

- **What it is:** Tables/relations/migrations, constraints, and **one source of truth** for entities (users, challenges, progress).
- **Why it matters:** Bugs become cheaper when the schema matches the domain language.

A6. HTTP API (or RPC) surface

- **What it is:** Stable routes, consistent error shapes, validation at the boundary, idempotency where needed.
- **Why it matters:** The frontend and any future mobile app depend on predictable contracts.

A7. Integrations & asynchronous work

- **What it is:** Calling Strava, **webhooks** (Strava pushes events), optional **cron**/background jobs, retries.
- **Why it matters:** External systems are flaky; you design for **eventual consistency** and **duplicate delivery**.

A8. Observability

- **What it is:** Structured logging, correlation, enough context to debug production without printing secrets.
- **Why it matters:** You will not reproduce every bug locally.

A9. Configuration & environments

- **What it is: Dev / preview / prod** differences, feature flags sometimes, **never** shipping private keys to the browser bundle.
- **Why it matters:** Most incidents are "wrong env var" or "secret leaked to client build."

A10. Security posture (minimum bar)

- **What it is:** HTTPS, sensible headers, validating inputs, rate limiting where justified, protecting admin surfaces, understanding XSS/CSRF tradeoffs for your session model.

- **Why it matters:** Not paranoia — **expected** for anything handling accounts.

A11. Legal & trust surface (when you handle people's data)

- **What it is:** Privacy policy, terms, consent capture where regulations or partners (Strava) expect it.
- **Why it matters:** Partners and users ask; engineering implements **what** was agreed in copy.

A12. Delivery pipeline & quality gates

- **What it is:** Build that fails fast, lint, tests, typecheck for critical paths, migration strategy for DB.
- **Why it matters:** This is how teams keep velocity without breaking prod every Friday.

A13. Documentation & operability

- **What it is:** README, runbooks for deploy/DB/webhooks, architecture notes for the next developer (you, in six months).
- **Why it matters: Bus factor** and onboarding cost are real costs.

Part B — How iRace meets each need (where to look)

This section ties **Part A** to **paths in this repo**. Treat it as a **guided tour**: open the file, skim imports and exports first, then read the happy path.

B1. Product shape & user journeys

- **Landing & marketing surface:** `src/components/Home/LandingPage.tsx` — first impression, CTAs, routes users into sign-up flows.
- **Core "race" experience:** `src/components/Race/RaceView.tsx` and siblings like `Leaderboard.tsx`, `RaceTrack.tsx`, `ActivityFeed.tsx`, `TauntsPanel.tsx` — the main value: progress, competition, and social taunts.
- **Create / join flows:** `src/components/Challenge/CreateChallenge.tsx`, `JoinChallenge.tsx`, `src/components/Challenges/MyChallenges.tsx` — lifecycle of a challenge from creation to listing.
- **Profile & settings:** `src/components/Profile/Profile.tsx` — account-adjacent settings (e.g. Strava connection, push).
- **Legal pages:** `src/components/Legal/PrivacyPolicy.tsx`, `TermsOfService.tsx` — static trust content linked from the shell.

Engineering idea: Routes in `src/App.tsx` mirror **user journeys**; when you add a feature, you usually add **a route + a page component + possibly an API route**.

B2. Client shell — routing, global state, failure handling

- **Route table:** `src/App.tsx` — central map of URL → screen; keeps navigation honest.
- **Auth state for the whole tree:** `src/context/AuthContext.tsx` — session token handling, session validation against the backend, Strava disconnect orchestration.

- **Cross-cutting UI feedback:** `src/context/ToastContext.tsx` — user-visible errors/success without duplicating toast wiring in every screen.
- **Fault isolation:** `src/components/ui/ErrorBoundary.tsx` — prevents a single component crash from blanking the entire app (paired with `src/main.tsx` for Strict Mode and dev diagnostics).
- **Shared API calling conventions (browser):** `src/lib/apiClient.ts` — how Bearer headers, JSON parsing, and HTTP errors are normalized for the frontend.

Engineering idea: Providers wrap the router so any deep component can consume auth/toast without prop drilling; **error boundaries** are insurance for unexpected React failures (different from API error toasts).

B3. Configuration the browser is allowed to know

- **API base URLs & canonical origins:** `src/config/urls.ts` — avoids hardcoding domains; important for OAuth redirect correctness across localhost vs production.
- **Endpoint constants:** `src/config/api.ts` — single place for `/api/...` paths the SPA calls.
- **Consent copy / versioning:** `src/config/consent.ts` — keeps compliance-related constants out of random components.

Engineering idea: Vite exposes only `VITE_*` variables to the client — see `.env.example` and README — so secrets stay server-side.

B4. Typed domain language (frontend)

- **Shared TypeScript models:** `src/types/index.ts` — Challenge, User, sports, etc. — aligns UI and services on the same words.

Engineering idea: Types are **documentation that the compiler enforces**; they pay off when you refactor.

B5. Service layer (browser talks to your backend)

- **Challenges CRUD/join/sync/taunts:** `src/services/challengeService.ts` — encapsulates HTTP details for challenge operations the UI needs.
- **Strava OAuth URL & Strava HTTP wrapper (client):** `src/services/stravaService.ts` — separates "how we call our API / Strava" from React components.
- **User operations:** `src/services/userService.ts` — same pattern for user-centric calls.
- **Heavier Strava sync / mapping:** `src/lib/strava-sync.ts`, `src/lib/strava.ts` — domain logic that is not just a thin HTTP wrapper (e.g. mapping activities, relevance filters).

Engineering idea: Components orchestrate; services perform I/O. That split makes testing and refactors easier than putting `fetch` in every button handler.

B6. Serverless API surface (Vercel)

Handlers live under `api/` — each file is an HTTP entrypoint. Examples:

- **Session introspection (Bearer token → user):** `api/auth/session/index.ts` — the backend counterpart to `AuthContext`'s session check.
- **Strava OAuth callback:** `api/auth/strava/callback.ts` — exchanges OAuth code for tokens **on the server**, then establishes your app session.
- **Strava actions (refresh, disconnect, sync, etc.):** `api/strava/[action].ts` — dynamic dispatch pattern for related Strava operations.
- **Strava webhook ingress:** `api/strava/webhook.ts` — asynchronous, signature-oriented endpoint (different failure modes than user clicks).
- **Challenges collection & sub-resources:** `api/challenges/index.ts`, `api/challenges/[id]/index.ts`, `api/challenges/user/[userId]/index.ts`, `api/challenges/taunts.ts` — REST-ish grouping by resource.
- **User:** `api/user/index.ts`
- **Web Push:** `api/push/[action].ts` — subscribe/unsubscribe/test channels.

Engineering idea: One folder per URL segment is a Vercel convention; it scales until you outgrow it — then you might introduce a router or monolith.

B7. Server-side domain logic (shared by API routes)

The `server/` directory is **not** another deployable — it is **libraries** imported by `api/` handlers:

- **Prisma lifecycle for serverless:** `server/prisma.ts` — create/dispose clients per invocation (important on lambdas).
- **Auth session helpers:** `server/authSession.ts` — centralizes how sessions are created/validated in code shared by routes.
- **Strava business logic:** `server/stravaHandlers.ts`, `server/stravaTokenRefresh.ts`, `server/stravaWebhookProcessor.ts` — keeps `api/` files thin.
- **Challenge lifecycle rules:** `server/challengeLifecycle.ts` — domain rules that should not be duplicated in React.
- **Logging:** `server/logger.ts` — structured, scoped logs for Node.
- **TLS edge case for dev:** `server/optionalInsecureTls.ts` — documented escape hatch; paired tests in `server/optionalInsecureTls.test.ts`.
- **Query helpers:** `server/vercelQuery.ts` — shared parsing/validation for HTTP concerns.
- **Web push config assembly:** `server/webPushConfig.ts` — server-side pieces for push.

Engineering idea: `api/` = HTTP adapter, `server/` = use-cases & policies. This mirrors what larger codebases call "service layer" or "domain layer."

B8. Database & migrations

- **Schema (source of truth):** `prisma/schema.prisma` — models like `User`, `Challenge`, `Participation`, `Session`, `PushSubscription`, etc.
- **Prisma client usage from the app:** `server/prisma.ts`; older/shared references may appear in `src/lib/db.ts` (know which runtime you are in — browser vs Node).

- **Operational docs:** `docs/DATABASE_SETUP.md`, `docs/DATABASE_VERCEL.md` — how to provision and connect (especially pooled vs direct URLs).

Engineering idea: Migrations are part of your deploy story — `vercel.json` build runs Prisma migrate/deploy so schema and code stay aligned.

B9. Integrations — Strava OAuth, REST, webhooks

- **OAuth redirect handler (server):** `api/auth/strava/callback.ts` — turns Strava's code into stored tokens and a **session** users carry in the SPA.
- **Ongoing Strava operations:** `api/strava/[action].ts` + `server/stravaHandlers.ts` — refresh, sync, disconnect patterns.
- **Webhook endpoint:** `api/strava/webhook.ts` + `server/stravaWebhookProcessor.ts` — asynchronous updates when Strava notifies you.
- **Webhook registration script (ops):** `scripts/register-strava-webhook.mjs` — automation that is not part of the user-facing bundle.
- **Strava submission / review docs:** `docs/STRAVA_SUBMISSION.md`, `docs/STRAVA_REVIEW.md`, `docs/STRAVA_WEBHOOK.md` — partner-facing and operator-facing knowledge.

Engineering idea: OAuth and **webhooks** are different species — OAuth is interactive; webhooks must be **verified**, **idempotent**, and safe under **retries**.

B10. Push notifications (optional product feature)

- **Browser subscription UX/helpers:** `src/lib/pushNotifications.ts` — service worker registration, subscribe/unsubscribe payloads.
- **Public VAPID key in frontend env:** documented in `.env.example` / `docs/WEB_PUSH.md`.
- **Server sending & persistence:** `api/push/[action].ts`, `server/webPushConfig.ts`, Prisma `PushSubscription` model — server stores endpoints and keys needed to send.

Engineering idea: Push ties **browser APIs**, **a server sender**, and **DB persistence** — a vertical slice, ideal for learning end-to-end.

B11. Security-related HTTP headers & hosting rules

- **Vercel routing & headers:** `vercel.json` — SPA fallback to `index.html` for non-API paths, security headers like `X-Frame-Options`, build pipeline hooks for Prisma.

Engineering idea: Headers are **cheap defense-in-depth**; they do not replace careful auth code.

B12. Testing & static quality

- **Unit tests (Vitest):** `shared/stravaSportType.test.ts`, `server/normalizeSports.test.ts`, `server/optionalInsecureTls.test.ts` — pure functions and critical edge cases.
- **Config:** `vitest.config.ts`
- **ESLint:** root config consumed by `npm run lint`

- **Separate TS project for serverless:** `tsconfig.api.json` — typechecks `api/` separately from Vite's browser TS config.

Engineering idea: Split typecheck targets when two runtimes (browser vs Node) have different constraints — reduces false positives and catches real issues.

B13. Tooling & scripts

- **DB maintenance / one-off migrations:** `scripts/setup-db.ts`, `scripts/migrate-db.ts`, `scripts/clean-db.ts`, `scripts/create-participation-table.ts` — operational scripts, not imported by the UI.
- **VAPID generation:** `scripts/generate-vapid-keys.mjs`

B14. Documentation set (meta)

- **README:** project overview, env vars, commands — first file a newcomer reads.
- **Architecture index:** `docs/ARCHITECTURE.md`
- **Deploy & domain:** `docs/DEPLOY_INSTRUCTIONS.md`, `docs/VERCEL_DOMAIN.md`
- **Audit trail of refactors:** `docs/AUDIT_CHANGES.md`

Part C — What to do first on a new project (order of work)

When you build **alone**, order reduces thrash. This is a **common** sequence; iRace broadly follows it even if history was messier.

1. **Clarify the job-to-be-done and one happy-path story**
   Example here: "Create challenge → share link → friend joins → both see leaderboard." Everything else is secondary.
2. **Pick runtime topology**
   Here: React SPA on Vercel + serverless `api/*` + Postgres. That choice drives where secrets live and how you connect to the DB.
3. **Model data**
   Define `User`, `Challenge`, `Participation` in `prisma/schema.prisma` (or equivalent). Without this, UI is fiction.
4. **Implement auth/session for *your* app**
   Strava OAuth gives you Strava access; you still need **your** session (`Session` model + `/api/auth/session`). See `api/auth/strava/callback.ts` and `api/auth/session/index.ts`.
5. **Build the API surface incrementally**
   Start with read paths, then mutations, then integrations.
6. **Build the UI against real endpoints**
   Services in `src/services/*` keep components thinner.
7. **Add async integrations**
   Webhooks (`api/strava/webhook.ts`) after manual sync works.

8. **Hardening: logging, tests on pure logic, headers, docs**
   Exactly what `README.md` "Before you push" describes.

Important early decisions (high leverage)

- **Where secrets live:** never `VITE_*` for `STRAVA_CLIENT_SECRET` — README spells this out.
- **Canonical URL for OAuth:** `src/config/urls.ts` — getting this wrong causes "works on localhost, fails in prod."
- **Session mechanism:** Bearer in `Authorization` + server lookup — see `api/auth/session/index.ts` and `AuthContext.tsx` — understand tradeoffs (discussed with your mentor re: `localStorage` vs cookies).

Part D — Thinking like an engineer (habits, not talent)

D1. Boundaries are a tool

Ask: **Which layer should know this?**

- **React component:** rendering and local UI state.
- **Service module (`src/services/*`):** HTTP to your API.
- **`api/*` handler:** HTTP status codes, auth extraction, input validation, calling `server/*`.
- **`server/*`:** business rules and Prisma.
- **`prisma/schema.prisma`:** invariants about data shape.

When you feel lost, **draw a box diagram** on paper: Browser → `api/` → `server/` → DB → Strava.

D2. Make invalid states unrepresentable (where cheap)

Enums for sports, typed route params, Prisma constraints — each removes a class of bugs.

D3. Optimize for the next reader

That is often **you**. Descriptive file names (`stravaWebhookProcessor.ts`) beat clever ones.

D4. Operability is a feature

If production fails and you cannot answer "what did the server think?", you will suffer. Logging in `server/logger.ts` exists for that.

D5. "Good enough security" still needs a threat model sketch

Not formal — five minutes: **What assets? What attackers? What's out of scope?** For iRace: session tokens, Strava tokens, user emails, challenge data — drives why webhook verification and secret hygiene matter.

Part E — Patterns you'll recognize at other companies (name mapping)

Idea	Often called	Where it shows up here
Page map	Route table	`src/App.tsx`
Global session	Auth provider / context	`src/context/AuthContext.tsx`
Design system atoms	UI kit	`src/components/ui/*`
API client wrapper	SDK / apiClient	`src/lib/apiClient.ts`, `src/services/*`
Use-cases	Application services	`server/*`
HTTP adapters	Controllers / handlers	`api/*`
ORM	Data access	Prisma (`prisma/schema.prisma`, `server/prisma.ts`)
Background/async ingress	Webhooks	`api/strava/webhook.ts`
Feature vertical slice	End-to-end module	Example: push — `src/lib/pushNotifications.ts` + `api/push/*` + Prisma model
Infra as config	Hosting config	`vercel.json`

Different companies use **Next.js**, **Rails**, **Django**, **Spring** — the **names** change; the **boxes** (UI, HTTP, domain, DB, integrations) stay.

Part F — A learning path through this repository

Suggested order for **self-study** (adjust to your curiosity):

1. **`README.md`** — commands and env — ground truth for running things.
2. **`src/App.tsx`** — see the whole product as routes.
3. **`src/context/AuthContext.tsx`** + **`api/auth/session/index.ts`** — understand **your** session model end-to-end.
4. **`api/auth/strava/callback.ts`** — see OAuth code exchange **on the server only**.
5. **`prisma/schema.prisma`** — skim models and relations.
6. **`api/challenges/index.ts`** + **`server/challengeLifecycle.ts`** — see how HTTP stays thin.
7. **`src/components/Race/RaceView.tsx`** + **`src/services/challengeService.ts`** — vertical slice from UI to API.
8. **`api/strava/webhook.ts`** + **`server/stravaWebhookProcessor.ts`** — async integration mindset.
9. **`docs/AUDIT_CHANGES.md`** — see how incremental refactors are documented.

Closing — your own “valid way”

Teams disagree on folder names (`services` vs `use-cases` vs `features`). What matters is **consistency** and **clear boundaries**. iRace separates `api/` (HTTP) from `server/` (domain) from `src/` (UI) on purpose — that is one coherent style you can reuse or react against when you define yours.

You do not need permission to adopt a structure — you need **a reason** each folder exists. This document gives you a **checklist of needs** and a **concrete map** of how one project satisfied them. Build the mental model first; the confidence of “I know where things go” follows.

Appendix A — Frontend audit changelog

This document is written to help a junior developer understand what we changed, why it matters, and which files were touched. We’ll keep app behavior stable while improving maintainability, readability, and correctness module-by-module.

Module 1 — App shell + error boundary

What changed

- **Added React Strict Mode** at the root render.
  - **File**: `src/main.tsx`
- **Fixed dev-only error details rendering** to use Vite’s environment flags.
  - **File**: `src/components/ui/ErrorBoundary.tsx`

Why it matters

- **Maintainability**: Strict Mode helps catch unsafe effects and deprecated patterns early while you’re developing (it intentionally double-invokes certain lifecycles in dev to surface issues).
- **Correctness**: Vite apps shouldn’t rely on `process.env.NODE_ENV` in the browser bundle; `import.meta.env.DEV` is the supported mechanism and avoids bundler surprises.

Touched files

- `src/main.tsx`
- `src/components/ui/ErrorBoundary.tsx`

Module 2 — Auth/session (single source of truth)

What changed

- Simplified auth state so it comes from **one place**:

- `session_token` in `localStorage`
- validated by calling `SESSION` (`/api/auth/session`)
- which returns `user` and `stravaTokens`

- Removed the older path that:
  - listened to `localStorage` keys like `strava_tokens` / `user_updated`
  - called Strava directly from the browser to build a profile
  - attempted client-side "rate limiting" for those calls
- Added abort-safe session check (prevents state updates after unmount).

Why it matters

- **Maintainability**: fewer moving parts (no duplicated "sources of truth" for user/tokens).
- **Security & boundaries**: avoids expanding the browser's responsibilities; the server already owns token refresh and session issuance.
- **Correctness**: fewer race conditions and fewer cases where UI can display stale tokens/user info.

Touched files

- `src/context/AuthContext.tsx`

Module 3 — Services layer: consistent API errors & auth headers

What changed

- Added a small shared API helper:
  - **File**: `src/lib/apiClient.ts`
  - Provides `ApiError` , `assertOk()` , `readJson()` , `readJsonOrNull()` , `getAuthHeader()` , `jsonAuthHeaders()` , and `jsonBearerHeaders()`
- Refactored services to use the shared helper and avoid repeated boilerplate:
  - **Files**:
    - `src/services/challengeService.ts`
    - `src/services/stravaService.ts`
- Extended the same patterns to **auth**, **push**, and `StravaAPI` (browser refresh path):
  - `src/context/AuthContext.tsx` — session and Strava disconnect use `getAuthHeader()` , `readJson()` , and `assertOk()` ; disconnect URL uses `API_BASE_URL` (see below).
  - `src/lib/pushNotifications.ts` — JSON + Bearer headers via `jsonBearerHeaders()` .
  - `src/lib/strava.ts` — refresh-token call uses `assertOk()` / `readJson()` and `API_BASE_URL` .
- **URL fix**: Several calls used `new URL('/something', CHALLENGES)` where `CHALLENGES` is `…/api/challenges` . A path starting with `/` resolves to the **site root**, so requests incorrectly

targeted `…/strava/…` and `…/challenges/taunts` **without** the `/api` prefix. Those endpoints now use `` `${API_BASE_URL}/…` `` explicitly.

Why it matters

- **Maintainability**: one place for "how we parse errors / JSON / auth headers".
- **Readability**: service methods become shorter and more uniform.
- **Correctness**: consistent error messages (prefer backend `error` field when available) and consistent status tracking via `ApiError.status`; API routes under `/api` are called reliably in dev and production.

Touched files

- `src/lib/apiClient.ts`
- `src/context/AuthContext.tsx`
- `src/lib/pushNotifications.ts`
- `src/lib/strava.ts`
- `src/services/challengeService.ts`
- `src/services/stravaService.ts`

Module 4 — Race view (data flow + correctness)

What changed

- Added a small date coercion helper so we never pass string dates directly into `date-fns/format()` (which expects a `Date`).
- Removed a duplicate "fetch the challenge again" inside the Strava-load path. Now the Strava refresh uses the already-loaded `challenge` data and only refreshes activities.
- Made the refresh handler use `try/finally` so the "refreshing" spinner state always resets even on errors.

Why it matters

- **Correctness**: backend returns JSON date strings; formatting them directly is easy to get wrong and can lead to runtime errors or weird date formatting.
- **Maintainability**: avoids repeated fetch logic and reduces effect complexity.
- **Performance**: less redundant network traffic (one fewer challenge fetch during Strava refresh).

Touched files

- `src/components/Race/RaceView.tsx`

Module 5 — Challenge flows (create/join/my challenges)

What changed

- **Join flow now actually uses the consent checkbox**:
  - Previously we always sent `challengeDataConsentAccepted: true` even if the user didn't check the box.
  - Now we require the checkbox and send the real value.
- **Centralized Strava OAuth URL building**:
  - Join now uses `getStravaAuthUrl(returnPath)` instead of manually constructing the URL.
- **Create challenge share link copy is now robust**:
  - Clipboard write is awaited and errors are handled with a toast + log.
- **My Challenges filters now have real types**:
  - Replaced generic `string` filters with narrow unions like `'all' | ChallengeStatus`, reducing bugs and improving autocomplete.

Why it matters

- **Correctness & compliance**: consent should reflect the user's action, not a hardcoded `true`.
- **Maintainability**: one OAuth URL builder means fewer subtle encoding/redirect bugs.
- **UX robustness**: clipboard can fail (permissions, insecure context); handling it prevents silent failures.
- **TypeScript maintainability**: typed filters reduce "stringly-typed" code and make refactors safer.

Touched files

- `src/components/Challenge/JoinChallenge.tsx`
- `src/components/Challenge/CreateChallenge.tsx`
- `src/components/Challenges/MyChallenges.tsx`

Module 6 — Profile + push notifications (UI robustness)

What changed

- Profile now uses the shared session-token helper instead of manually reading `localStorage` in multiple places.
- Push notification calls now use `API_BASE_URL` (centralized) instead of recomputing the base URL per handler.

Why it matters

- **Maintainability**: consistent session-token access means fewer “where does auth come from?” surprises.
- **Readability**: less repeated glue code inside UI event handlers.

Touched files

- `src/components/Profile/Profile.tsx`

Appendix B — Architecture notes

API routes (Vercel)

Handlers under `api/` are **TypeScript** (`VercelRequest` / `VercelResponse` from `@vercel/node`). Shared helpers live in `server/` (mostly JavaScript today). Local API typecheck: `npm run typecheck:api` .

Database: Prisma

The app uses [Prisma]() for the PostgreSQL schema, migrations, and all runtime database access from API routes and server workers.

- Per-request / serverless handlers use `createFreshPrismaClient()` from `server/prisma.ts` and `$disconnect()` in `finally` blocks.
- `src/lib/db.ts` and other app code use the shared client where appropriate; reserve `$queryRaw` only for queries that cannot be expressed cleanly in the Prisma API.

Logging

Scoped loggers live in `server/logger.ts` (Node) and `src/lib/logger.ts` (browser). See inline comments for `LOG_DEBUG` on the server.

Shared modules

- `shared/stravaSportType.js` — maps Strava activity `type` / `sport_type` strings to Prisma `Sport` enum values. Imported from `server/*` , Vite (`src/`), and covered by tests.

Tests

- **Vitest** (`npm run test`): `shared/*.test.ts` , `server/*.test.ts` — pure helpers (`normalizeSports` , Strava sport mapping, opt-in TLS env). API routes stay integration-tested manually or via staging until you add HTTP-level tests.

Docs index

- Deployment / DB: `DATABASE_VERCEL.md`, `VERCEL_DOMAIN.md`
- Strava: `STRAVA_WEBHOOK.md`, `STRAVA_SUBMISSION.md`, `STRAVA_REVIEW.md`
- Push: `WEB_PUSH.md`